There are a number of patterns you can use to refactor your codebase; using the right one will depend on the context of your changes. I've seen the pattern illustrated in figure 8.20, called "surround and starve," work well for moving from large monolithic codebases to smaller, more modular, and easier-to-maintain codebases, as alluded to in this case study. There are entire books dedicated to refactoring. Two good ones are *Refactoring: Improving the Design of Existing Code* by Martin Fowler (Addison-Wesley Professional, 1999) and *Refactoring to Patterns* by Joshua Kerievsky (Addison-Wesley Professional, 2004).

After breaking down their development plan, the team realized that they could refactor as they went along without adding much additional time to their current updates. By focusing on new code that was decoupled from the problematic code, they could ensure that they had good code coverage and clean code from the start. They created a new project in SonarQube so they could monitor their new projects and ensure they had good code coverage and were following standards.

Every time they got a new task that would be tagged "cam," they started developing with their new approach. At first, development time for new tasks went up by a few days, but after only five releases they started to see the benefits of this approach, as shown in figure 8.21.

Figure 8.21. After five releases development time was reduced by over 60%.